

Chaos Language Algorithm: Particular Algorithm Version Used in the Recorded Tests

Chaos Language Algorithm Project

2026-07-10

1 Executive summary

The recorded CLA tests used the local pure-Python `chaoslang` prototype, not a Hyperon runtime and not the full theoretical CLA specification. The tested algorithm is best described as:

A deterministic, exact-reconstructing symbolic-stream grammar learner that first converts a continuous trajectory into fixed rectangular M1 symbols, then greedily accepts chunk and optional category edits that strictly reduce a simple two-part MDL-like score.

There are two closely related tested versions:

1. The 256-symbol attractor baseline and Lorenz-96 scaling records used the M1-control CLI with the then-current default CLA path: fixed-partition M1 symbolization, n-gram chunk proposals, exact-context category proposals, approximate MDL scoring, and eight greedy iterations.
2. The later 1024-step by 20D smoke run used local commit `b344aa3c1a4b5a5eeef93fc2ea239a7f75c97429`, plus one uncommitted CLI-only modification exposing miner/category options and fit timing. The run selected `--miner suffix_trie` and `--category-method js`. In that commit, the default `NGramPatternMiner` implementation had already been replaced internally by a bounded suffix trie.

The core learning semantics are the same across both: exact reconstruction is asserted after edits; chunks are sequential productions; categories are substitutable sets whose occurrences retain the observed member as $M[v]$; and candidate edits are accepted only when the score decreases.

2 Code provenance

At inspection on 2026-07-10, local `main` and `agent/suffix-trie-miner` both pointed to `b344aa3`; `origin/main` pointed to `847390c`. The local tree had one modified file, `src/chaoslang/benchmarks/m1_controls`. That modification was not a learner semantic change; it added CLI flags `--miner`, `--category-method`, `--js-threshold`, and `fit_wall_time_seconds` reporting.

3 Algorithm actually run

3.1 Inputs

For the benchmark tests, the learner received a finite tuple of symbolic tokens. For attractor controls this tuple was produced as follows:

Component	Version/evidence	Role
Repository	bgoertzel-sing/chaos-language-algorithm	Public GitHub repo
Local path	projects/chaos-language-algorithm/repos/chaoslang	Tested source tree
Package	chaoslang version 0.1.0	Pure-Python prototype
Sprint-1 commit	4a7399c	Symbolic-string MVP
M1 controls commit	cab962b lineage	Attractor generators and M1 symboliz
Dim-scaling records	adc005f/1501346 lineage	Lorenz-96 dim 8–24 records
JS categories	d1fd8f9	JS context clustering path
Persistence	e64b1d0	JSON/replay support, not central to t
Suffix-trie commits	97cfb4f, b344aa3	Bounded trie miner
1024x20D run	b344aa3 + dirty CLI mod	Exact recorded smoke run

Table 1: Source versions relevant to the tests.

1. Generate a deterministic trajectory using local stdlib numerical code.
2. Discard an initial prefix.
3. Infer per-axis min/max bounds from the retained trajectory unless explicit bounds are supplied.
4. Divide each axis into equal-width bins.
5. Convert each scalar or vector point into one compound token, e.g. `m1:d0i|d1j|...`

This is M1 fixed rectangular symbolization. It is not a learned partition, not a dimension-reduction method, and not a semantic OmegaSim state abstraction. For dimension D with four bins per axis, the direct compound-symbol space has up to 4^D labels.

3.2 Initial grammar state

The initial state is a corpus and parse equal to the raw symbol stream:

```
state = GrammarState.initial(stream)
parse = corpus.symbols
productions = {}
categories = {}
```

The main invariant is:

Expanding the current parse through the current grammar must exactly reproduce the original corpus symbols.

3.3 Chunk proposal path

A chunk is a sequential nonterminal production, e.g. `N7 -> a b c`. Candidate chunks are repeated contiguous parse blocks. The tested default parameters were:

- minimum n-gram length: 2
- maximum n-gram length: 5 in `CLA.simple`; the underlying bounded-trie class default is 6 when constructed directly
- minimum non-overlapping uses: 2

- greedy iterations in recorded benchmarks: 8

For older 256-symbol records, the miner was the then-current n-gram path. In the latest inspected code at `b344aa3`, `NGramPatternMiner` is implemented by a depth-bounded suffix trie. A separate legacy wrapper class `SuffixTrieMiner` remains selectable through `miner=suffix_trie` and was selected in the 1024x20D run.

Candidate chunk proposals are sorted deterministically and named `N0`, `N1`, etc. Accepted chunks replace non-overlapping occurrences in the parse. Productions whose use count drops below two are pruned and inlined, while exact reconstruction is reasserted.

3.4 Category proposal path

A category is not a sequential production. It is a substitutable set of tokens, with each parse occurrence preserving the observed member:

```
M[x] expands as x
M[y] expands as y
```

Thus categories can compress context-substitution patterns without losing exact reconstruction. Two category proposal mechanisms exist:

1. **Exact context categories:** group tokens with identical immediate left/right context signatures.
2. **Jensen-Shannon categories:** group tokens whose left/right context histograms cluster below a JS-divergence threshold.

The 1024x20D smoke run selected JS categories with threshold 0.1. It exercised this path but accepted no category edit, so that run does not demonstrate a compression benefit from JS clustering.

3.5 Scoring and search

The search is greedy over candidate edits. At each iteration:

1. score the current state;
2. enumerate chunk proposals;
3. optionally enumerate category proposals;
4. apply each proposal to a copied state;
5. score each candidate;
6. choose the candidate with the lowest strictly smaller score;
7. if no candidate improves the score, stop.

Ties are broken deterministically by proposal class name and representation. Although the API accepts a seed and constructs a private RNG, the inspected fit loop does not use randomness in proposal selection.

The scorer is a simple two-part MDL-like proxy in token-count units, not a calibrated bit code:

- each ordinary parse entry costs 1.0 data unit;
- each chunk production costs $0.8 + \text{rhs length}$ model units;
- each category costs $14.0 + \text{member count}$ model units;
- each edit-log entry costs 0.1 model units;
- a category occurrence costs $1 - \log_2 p(\text{member} \mid \text{category})$ data units.

This score is sufficient for strict local improvement tests, but should not be interpreted as absolute entropy or bits per symbol.

4 Version-by-test matrix

Test group	Symbolization	Learner path	Notes
Scalar/3D baselines	M1, 4 bins	n-gram chunks + exact categories	256 symbols, 8 iterations
Lorenz-96 dims 8–24	M1, 4 bins	n-gram chunks + exact categories	256 symbols, seed 0
1024x20D smoke	M1, 4 bins	suffix-trie chunks + JS categories	b344aa3 + CLI mod
Unit tests	direct symbols + M1	all public paths	71 tests passing on 2026-07-10

Table 2: Particular CLA version paths used by the recorded tests.

5 Not used in these tests

The following are part of the broader CLA/project plan, but were not the algorithm actually validated in the recorded experiments:

- Hyperon or Atomspace runtime execution;
- learned dimension reduction or semantic state projection;
- McBride-derivative pre-ranking;
- joint chunk/category proposals based on emergent synergy;
- quantale-generalized scoring;
- natural-gradient or beam search;
- held-out prediction or attractor classification;
- actual OmegaSim trace ingestion.

6 Current verification

The current local tree was rechecked while preparing this document:

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
```

Result: 71 tests passed in 0.901 seconds. The test count differs from the earlier summary’s 69 because the currently inspected tree includes all tests visible in the local working copy on 2026-07-10.

7 Bottom line

The algorithm version tested so far is a conservative MVP: fixed-partition trajectory symbolization plus deterministic exact-reconstructing grammar induction with chunk rules, member-preserving categories, and a simple MDL-like greedy acceptance rule. The 1024x20D test used the suffix-trie-enabled local version and JS category proposal path, but its accepted grammar still consisted only of chunk edits. Stronger CLA ideas from the formal plan remain engineering targets rather than validated components.